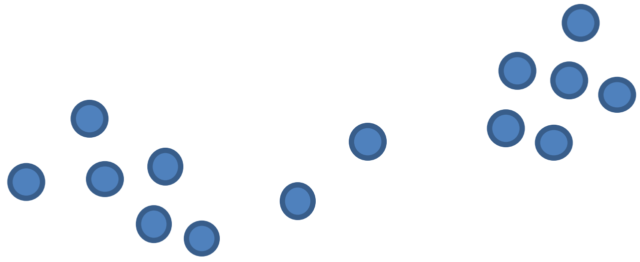
Clustering

Lee-Ad Gottlieb

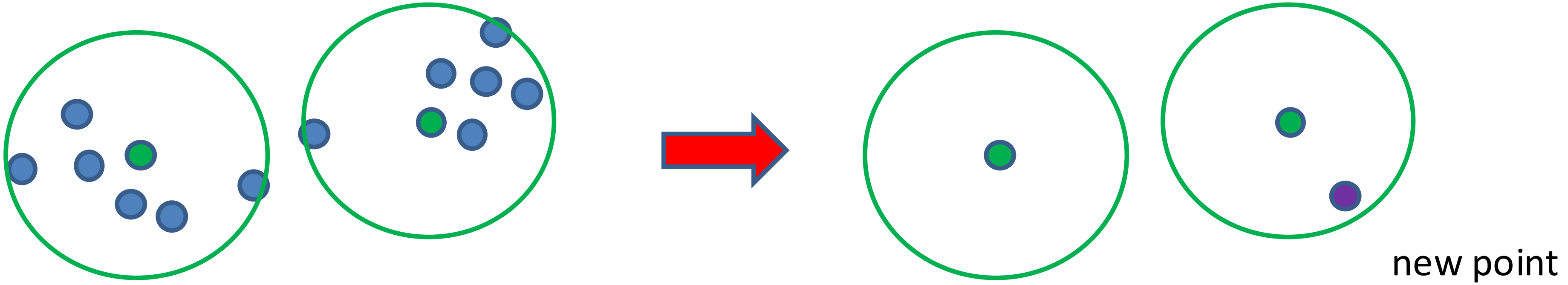
Clustering

- Supervised learning: sample points have labels
- Unsupervised learning: no labels!
  - Still possible to learn
  - For example, cluster these points into two groups



k-clustering

- Goals for clustering:
  - Learning: A new point can be assigned a group
  - Compression: May only need to retain center-set K, instead of all S
    - Better generalization bounds
  - Run-time: nearest neighbor search on base set K, not S



k-clustering

- Generalization bounds via compression.
  - What's the VC-dimension of learning (classification) via k-clustering?
  - Choose a center set K of size k from S: $\sim|S|^k$ possibilities.
  - If d points are shattered,

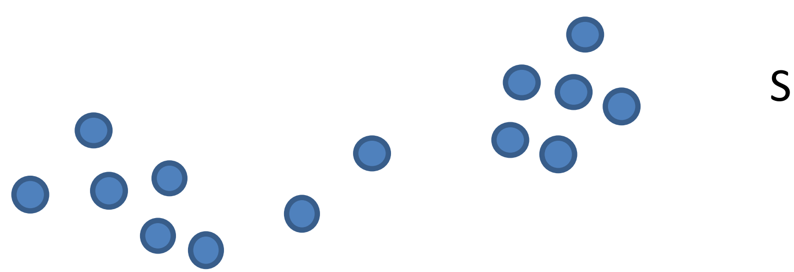
$$d^k \geq 2^d$$

$$k \log d \geq d$$

$$d = O(k \log k)$$

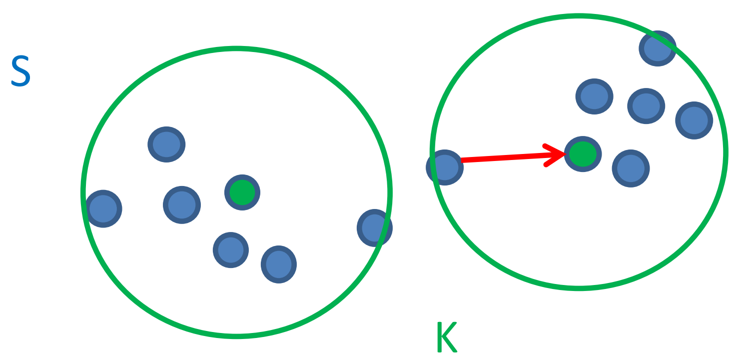
Clustering algorithms

- Problem: Given a sample S of n points and a parameter k
  - cluster S into k groups.
  - Question of measure: What’s a good clustering?
  - Different measures: **k-center, k-median, k-means**



k-center clustering

- **k-center**:
  - Problem: Given a sample S of n points and parameter k, cluster S into k groups
  - Measure: Choose a set of k centers $K \subset S$ that minimize $\max_{v \in S} d(v, K)$
    - Distance from each $v \in S$ to closest point in K



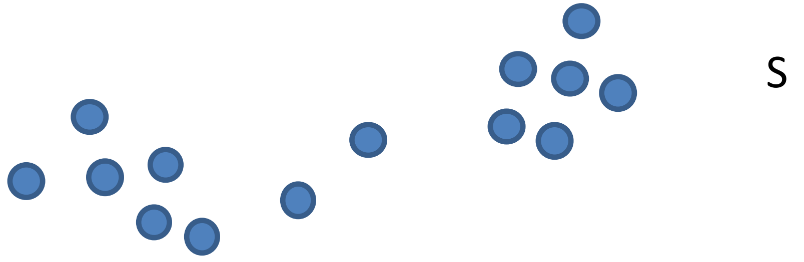
k-center clustering

- **k-center**: In the previous slide we assumed centers $K \subset S$
  - This is the discrete version (K subset of S)

- Non-discrete version:
  - centers chosen from ambient Euclidean space

k-center clustering

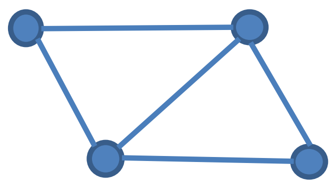
- Exact algorithm for discrete k-center:
  - Brute force: try all sets of k points in S
  - in $O(|S|^k)$ time
- Can we do better?

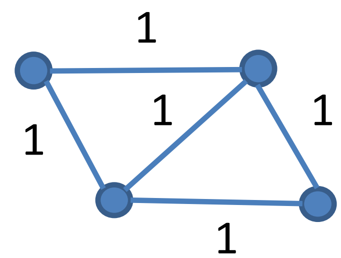


k-center clustering

- Finding optimal k-center clustering is NP-hard
  - When k is large
  - Also NP-hard to approximate radius within factor $2 - \epsilon$

Graph for Dominating Set problem



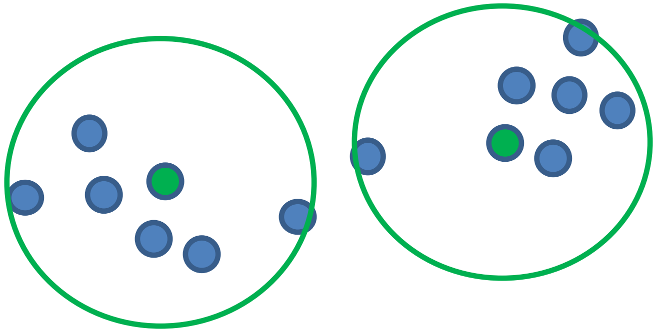


Point set For k-center clustering

- Proof (of both statements): reduction from **Minimum Dominating Set**
  - Given graph G=(V,E), find minimum sized subset $V' \subset V$
  - Such that any vertex $v \in V - V'$ is adjacent to some vertex in V'
  - NP-hard problem

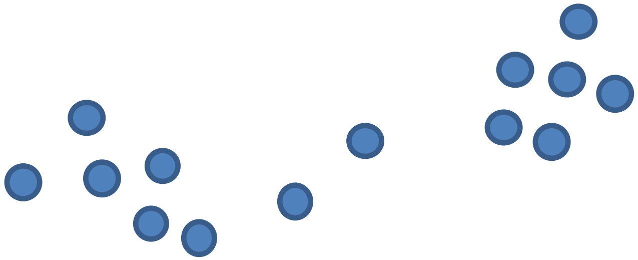
k-center approximation

- K-center is NP-hard to solve exactly
- Two possible polynomial-time approximation algorithms for the discrete case:
  - 1. Optimal radius, but k(ln n) centers
    - NP-hard to get less than k(ln n) centers
  - 2. k centers, but larger radius
    - NP-hard to do better than twice the radius



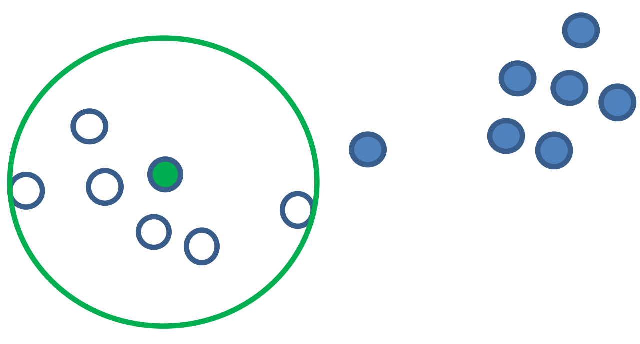
k-center approximation

- First approximation algorithm:
  - Optimal radius, but (k ln n) centers
- Greedy algorithm:
  - Assume optimal radius r is known (n^2 guesses)
  - Take center which covers most points
  - Remove covers points and repeat



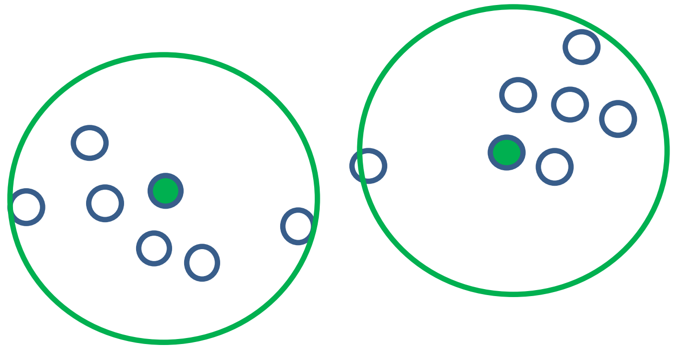
k-center approximation

- First approximation algorithm:
  - Optimal radius, but (k ln n) centers
- Greedy algorithm:
  - Assume optimal radius r is known (n^2 guesses)
  - Take center which covers most points
  - Remove covers points and repeat



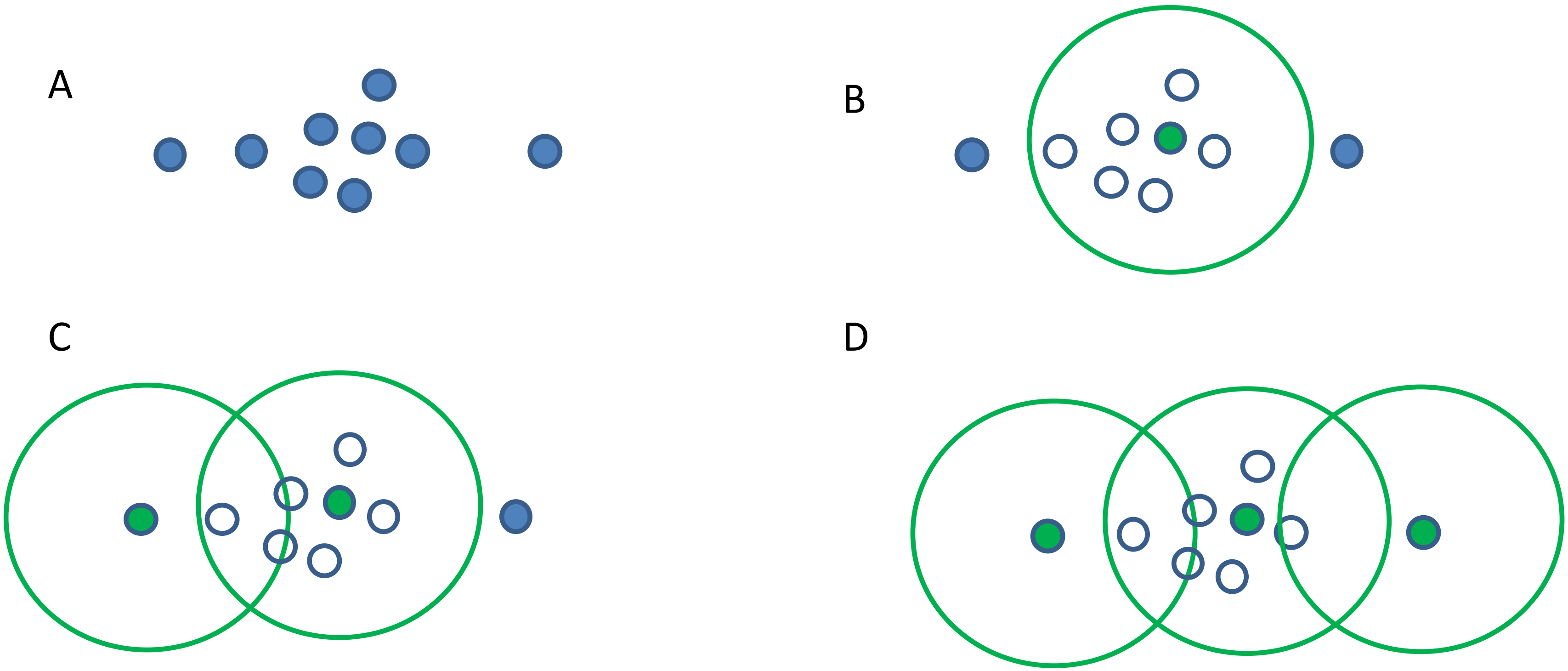
k-center approximation

- First approximation algorithm:
  - Optimal radius, but (k ln n) centers
- Greedy algorithm:
  - Assume optimal radius r is known (n^2 guesses)
  - Take center which covers most points
  - Remove covers points and repeat



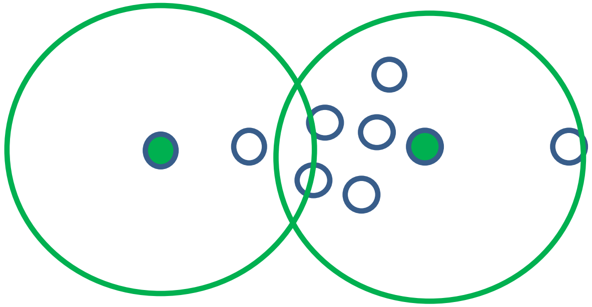
k-center approximation

- Greedy not necessarily optimal
  - Greedy:



k-center approximation

- Greedy not necessarily optimal
  - Optimal:

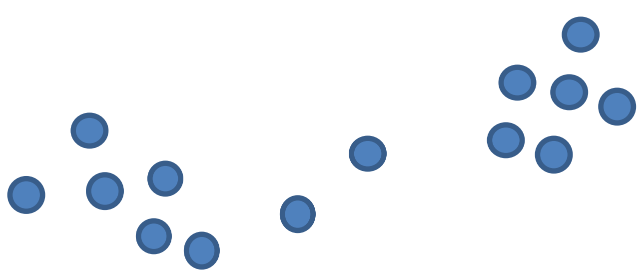


k-center approximation

- Standard greedy analysis:
  - k centers cover S, and also any subset of S
  - Some center covers a $\frac{1}{k}$ fraction of S or its subset
- At every step, a $\frac{1}{k}$ fraction of points are covered.
  - After i iterations, $\left(1 - \frac{1}{k}\right)^{i} n \leq e^{-\frac{i}{k}} n$ points remain
- Max i = k ln n iterations.
  - So (k ln n) centers, instead of k

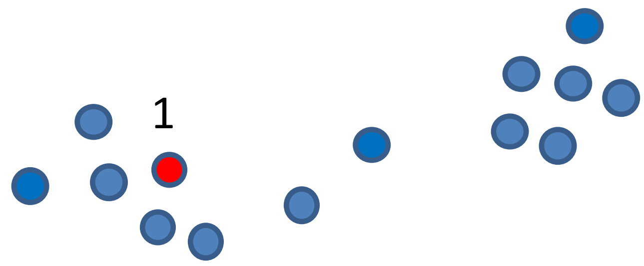
k-center approximation

- Second approximation algorithms:
  - k centers, but larger radius
- Greedy algorithm:
  - Choose the farthest uncovered point



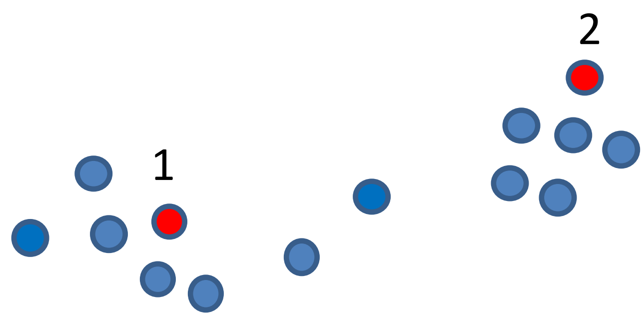
k-center approximation

- Second approximation algorithms:
  - k centers, but larger radius
- Greedy algorithm:
  - Choose the farthest uncovered point
  - Example: k=4



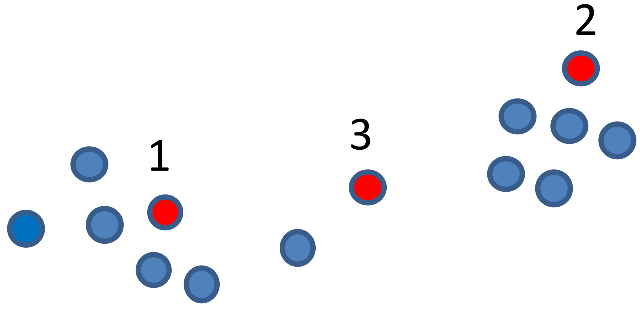
k-center approximation

- Second approximation algorithms:
  - k centers, but larger radius
- Greedy algorithm:
  - Choose the farthest uncovered point
  - Example: k=4



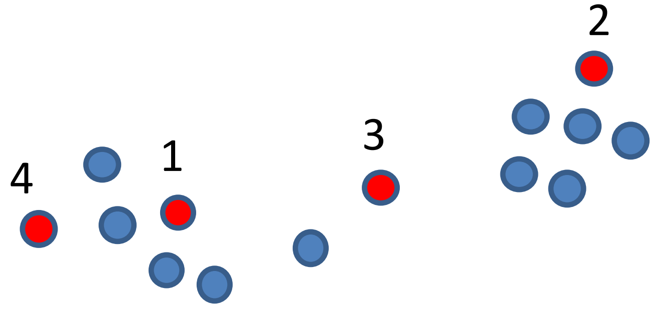
k-center approximation

- Second approximation algorithms:
  - k centers, but larger radius
- Greedy algorithm:
  - Choose the farthest uncovered point
  - Example: k=4



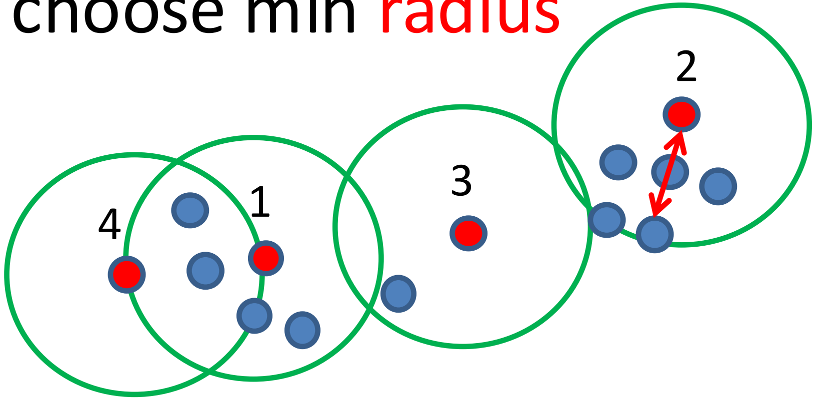
k-center approximation

- Second approximation algorithms:
  - k centers, but larger radius
- Greedy algorithm:
  - Choose the farthest uncovered point
  - Example: k=4



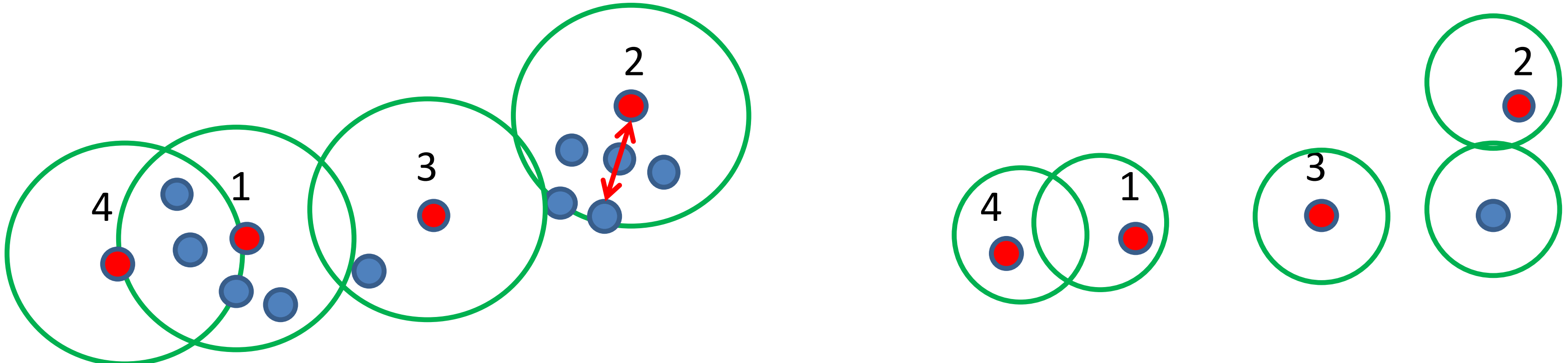
k-center approximation

- Second approximation algorithms:
  - k centers, but larger radius
- Greedy algorithm:
  - Choose the farthest uncovered point
  - Example: k=4
  - Now choose min radius



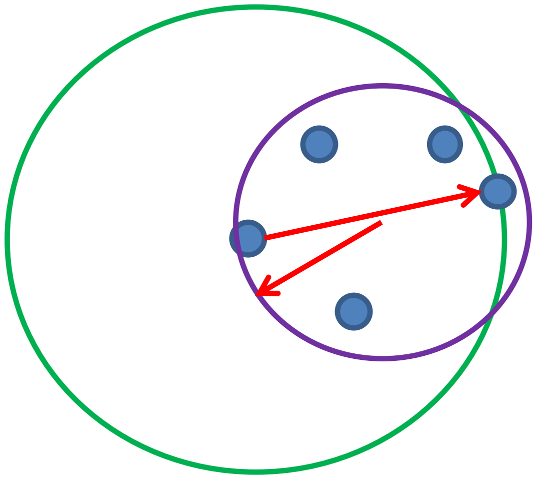
k-center approximation

- Greedy algorithm:
  - Choose the farthest uncovered point
- **Claim**: Final radius r at most 2r*
  - Where r* is optimal radius
  - At least k+1 balls of radius less than r/2 needed to cover all points.
  - So r/2 ≤ r* ≤ r



k-center approximation

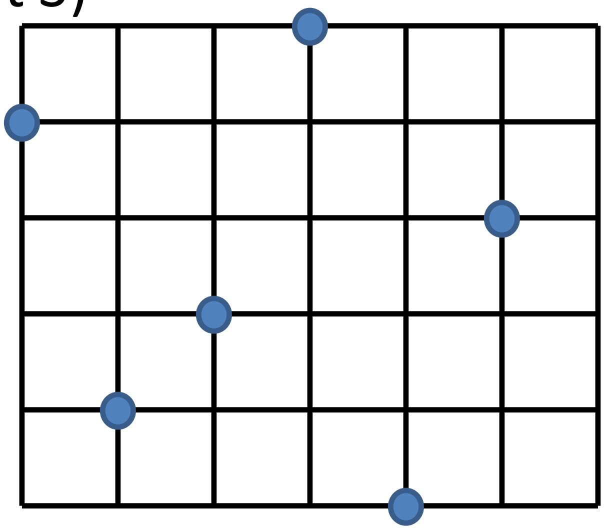
- What about the non-discrete case?
  - (Centers come from ambient space, not S)



- Can just solve the discrete case
  - Lose a factor 2 in the radius.

k-center approximation

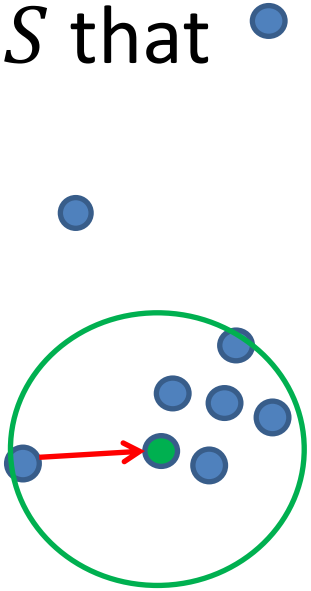
- What about the non-discrete case?
  - (Centers come from ambient space, not S)



- Can also take an $\frac{\epsilon}{\sqrt{d}}$-grid
  - $\left(\sqrt{d}/_{\epsilon}\right)^{d}$ gridpoints which can be centers
  - Can preprocess: JL reduces dimension to $\mathrm{d} = \frac{\log n}{\epsilon^2}$
    - But runtime still large

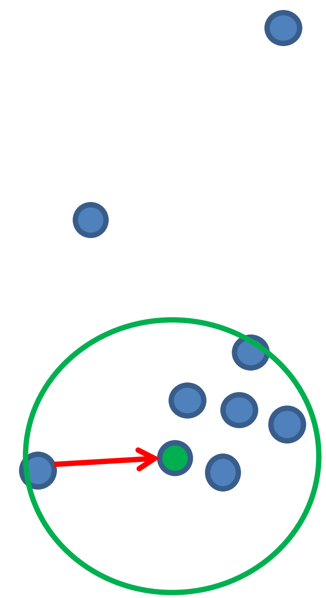
Clustering

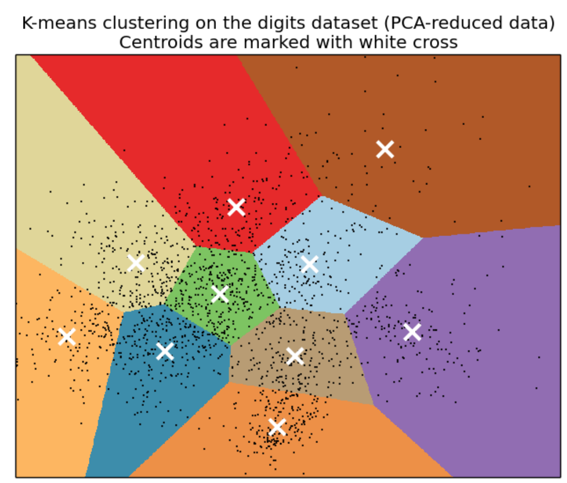
- **k-center**: Choose a set of k centers $K \subset S$ that minimize $\max_{v \in S} d(v, K)$
  - Distance from v to closest point in K
  - Problem: Not robust to outliers
- Other measures
  - **k-median**: Choose K to minimize $\frac{1}{|S|} \sum_{v \in S} d(v, K)$
  - **k-means**: Choose K to minimize $\frac{1}{|S|} \sum_{v \in S} d^2(v, K)$



Clustering

- k-center: max (ℓ_∞)
  - Balls
- k-median: average (ℓ_1)
  - Voronoi diagram
- K-means: average-squared (ℓ_2)
  - Voronoi diagram



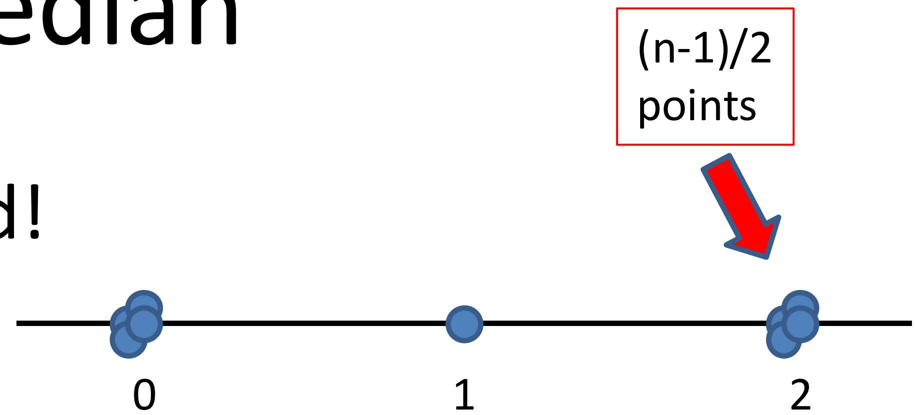


K-median

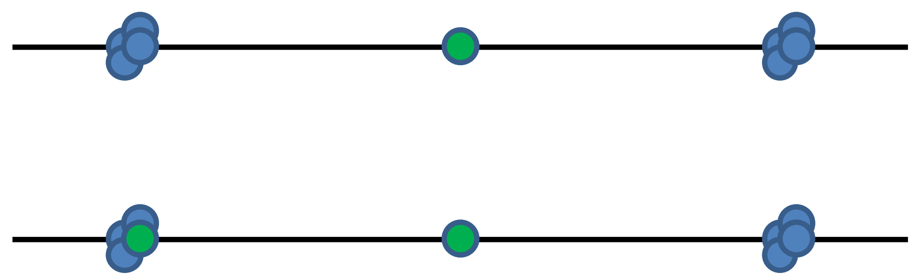
- How about the following greedy algorithm for k-median?
  - 1. $K \leftarrow \emptyset$
  - 2. While $|K| < k$
  - 3. Find $p \in S$ for which cost($K \cup \{p\}$) is minimal
  - 4. $K \leftarrow K \cup \{p\}$

K-median

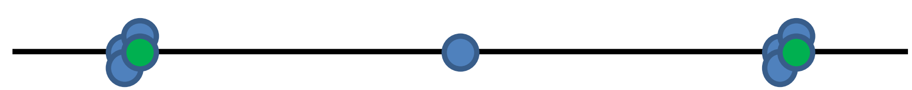
- Greedy algorithm is bad!
  - Proof: n points, k=2



- Greedy:
  - Cost: $\frac{n-1}{2}$



- Opt:
  - Cost: 1



K-median

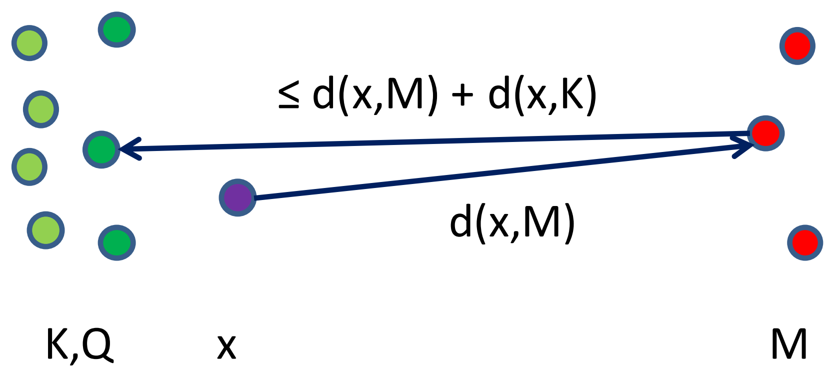
- How about the following reverse-greedy algorithm for k-median?
  - 1. $K_n \leftarrow S$
  - 2. For i=n,…,k+1
  - 3. Find $p \in K_i$ minimizing cost($K_i \setminus \{p\}$)
  - 4. $K_{i-1} \leftarrow K_i \setminus \{p\}$

K-median

- The reverse-greedy algorithm was suggested by Amos Fiat.
- Chroback, Kenyon, Young:
  - reverse-greedy gives a logn-approximation.

Preliminary

- Consider sets K and M, and let $Q \subset K$ be the nearest neighbors of M in K
- Claim: For any x,
  - $d(x,Q) \leq 2d(x,M) + d(x,K)$
- Proof: triangle inequality



K-median

- Proof:
  - Let M be the optimal solution.
  - Let $Q_i \subset K_i$ be the closest points to M in K_i
- $cost(K_{i-1}) - cost(K_i)$
- $= [\min_{r \in K_i} cost(K_i \setminus \{r\})] - cost(K_i)$

  definition
- $\leq [\min_{r \in K_i \setminus Q_i} cost(K_i \setminus \{r\})] - cost(K_i)$

  smaller set
- $\leq \frac{1}{|K_i \setminus Q_i|} \sum_{r \in K_i \setminus Q_i} [cost(K_i \setminus \{r\}) - cost(K_i)]$

  Min ≤ Average
- $\leq \frac{1}{i-k} \sum_{r \in K_i \setminus Q_i} [cost(K_i \setminus \{r\}) - cost(K_i)]$

  $|Q_i| \leq k$
- $\leq \frac{1}{i-k} [cost(Q_i) - cost(K_i)]$

  Removed at once
- $\leq \frac{2}{i-k} cost(M)$

  Previous slide

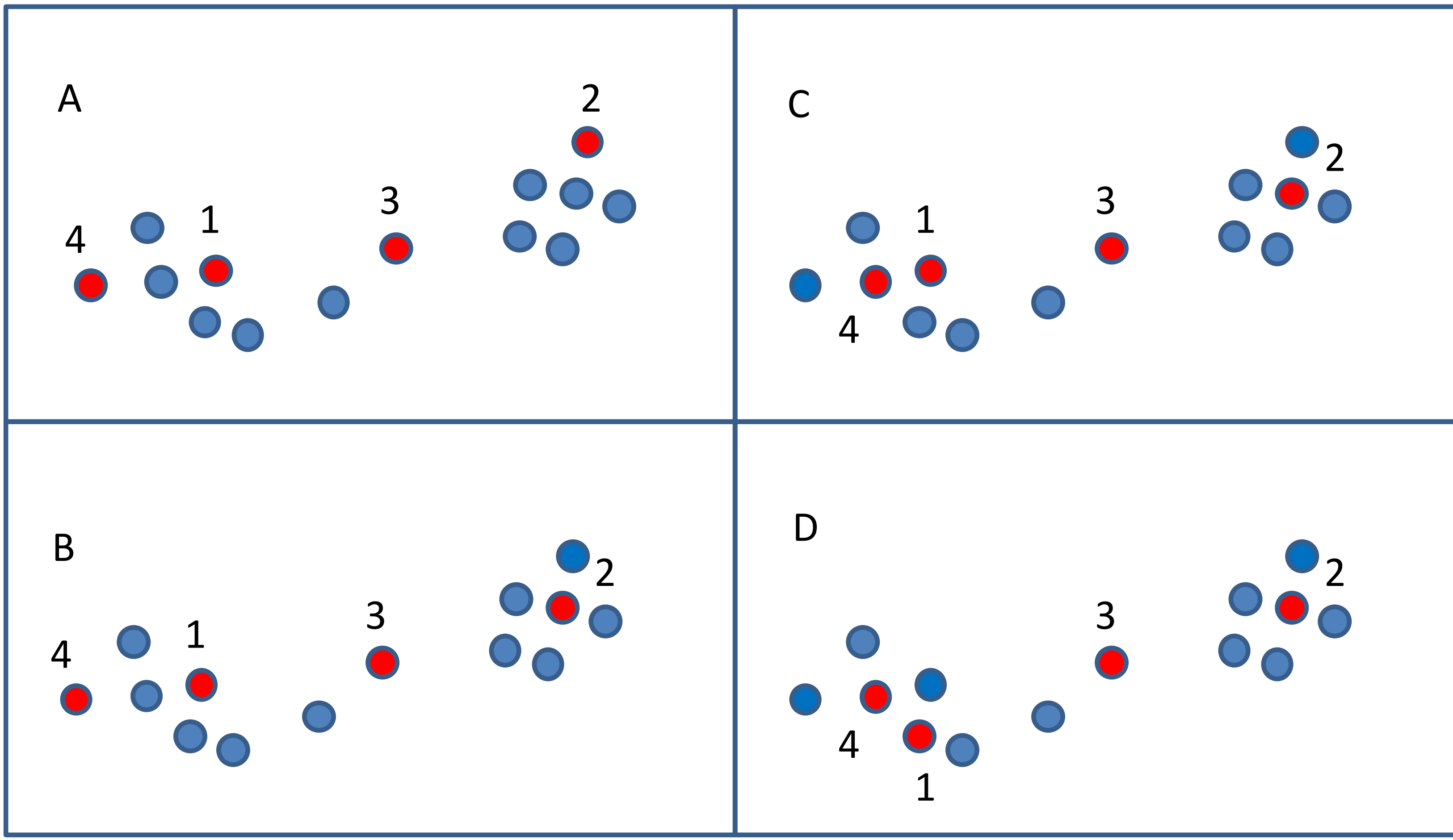
K-median

- Sum of cost increases is a Harmonic series
$$\frac{1}{n}+\frac{1}{n-1}+\cdots=O(\log n)$$
- So we have logn-approximation

K-median

- Another popular algorithm for k-median:
- Local search:
  - 1. Start with arbitrary set K
  - 2. Swap some $p \in K$ with some $q \in S \setminus K$ which maximizes reduction in cost
  - 3. Repeat 2 until no improvement possible
- Local minimum: 5-approximation
- Run time: ?
  - Can stop when improvement is below some threshold

Local Search (1-swap)



k-means algorithm

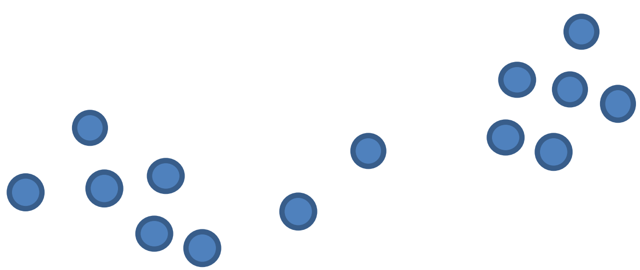
- Algorithm for non-discrete k-means
  - Variant of Lloyd's algorithm

- 1. Pick k centers arbitrarily
  - Many papers on this choice - *seeding*
- 2. Assign each point in S to closest center
- 3. For each cluster C, compute centroid
  - Centroid = $\frac{1}{|C|}\sum_{x \in C} x$
- 4. The centroids are the new centers: Assign each point in S to closest centroid
- 5. Repeat 2,3,4 until no change

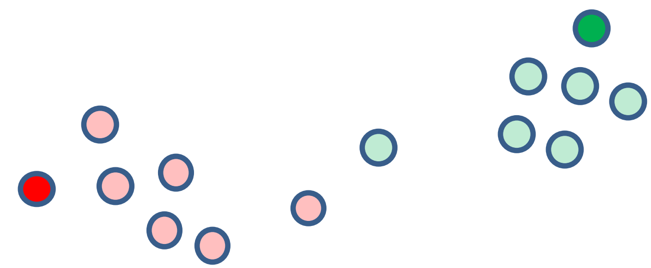
k-means algorithm

- Example

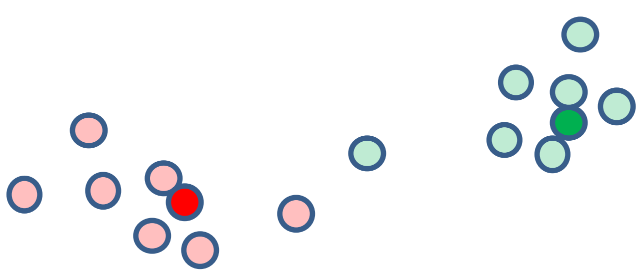
S



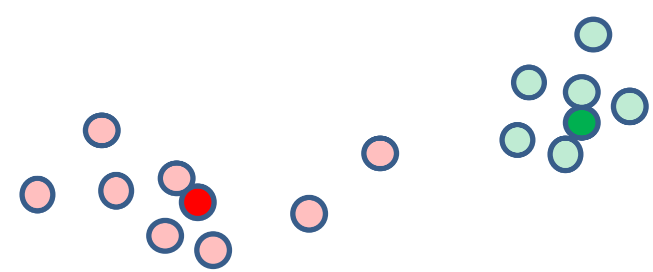
Choose 2 centers



Compute centroids



Reassign points

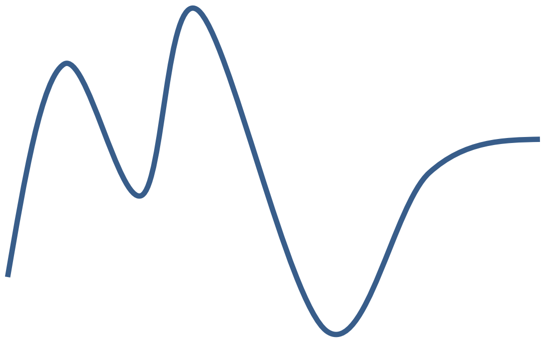


k-means

- What guarantees does this algorithm have?

- Iterations: worst-case $2^{\sqrt{n}}$
  - In practice, average case runtime much better

- Quality of solution: local minima
  - Why seeding is important



Where algorithm fails

- Point set

- Stable solution

- Opt